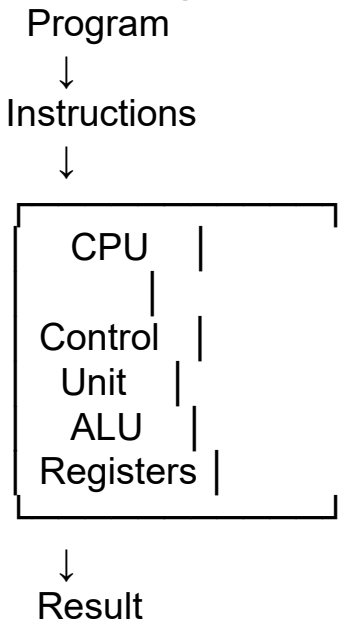
Assignment No – I

1. Write a short note on Evolution and history of computation system.
2. Define major differences between Computer architecture and computer organization.
3. State Eight great ideas in computer architecture.
4. Draw block diagram of computer system and explain in brief.
5. Write a short note on Computer Memory system.
6. What is Operating system and list its main function.
7. Write a short note on Basic instruction execution cycle in brief.
8. Write a short note on Number system.
9. Solve Below Conversions
    a) Find the decimal equivalent of $(140)_8$
    b) Convert $(50)_{10}$ to Binary

What is a Processor?
Simple definition
A **processor (CPU – Central Processing Unit)** is the main component of a computer that **executes instructions and performs operations on data**.
In simple words:
Processor = The part of the computer that reads instructions, processes data, and produces results.
Basic working



Main parts of a processor

Component	Function
Control Unit (CU)	Controls and coordinates operations
ALU	Performs arithmetic and logical operations
Registers	Temporarily store data and instructions
Cache	Stores frequently used data/instructions for fast access

imple example

Suppose we want to calculate:

10 + 20

The processor needs to:

1. Get the instruction.
2. Get the required data.
3. Perform addition.
4. Store the result.

10 + 20

↓

Processor

↓

ALU performs addition

↓

30

What is an Instruction?

How does the processor know what work it has to perform?

The answer is:

The processor performs instructions.

Definition

An **instruction** is a command given to the processor that tells it **what operation to perform and, when required, on which data**.

For example:

ADD R1, R2

This tells the processor to perform an **addition operation** using the specified registers.

Computer Organization and Architecture (CSE1003)

Syllabus

Module I: Introduction to Computer Architecture & Data Representation

Module II: Processor Architecture & Instruction-Level Parallelism

Module III: Memory Systems & Cache Organization

Module IV: I/O Systems

Module V: Multiprocessors and Parallel Processing



MODULE II:

Processor Architecture & Instruction-Level Parallelism

Prepared by- Ashwini Dhumal

Module II: Processor Architecture & Instruction-Level Parallelism

- **Instruction Set Architecture (ISA):** Concept of ISA, Types of instructions, Instruction formats (overview).

- **Register Transfer Language and Microoperations:** RTL notation,Register transfer and arithmetic microoperations.

- **Processor Organization:** ALU and its functions, Stack-based and general-purpose register architectures, Procedure calls and stack usage.

- **RISC and CISC Architectures:** Characteristics of RISC, Characteristics of CISC, Comparison.

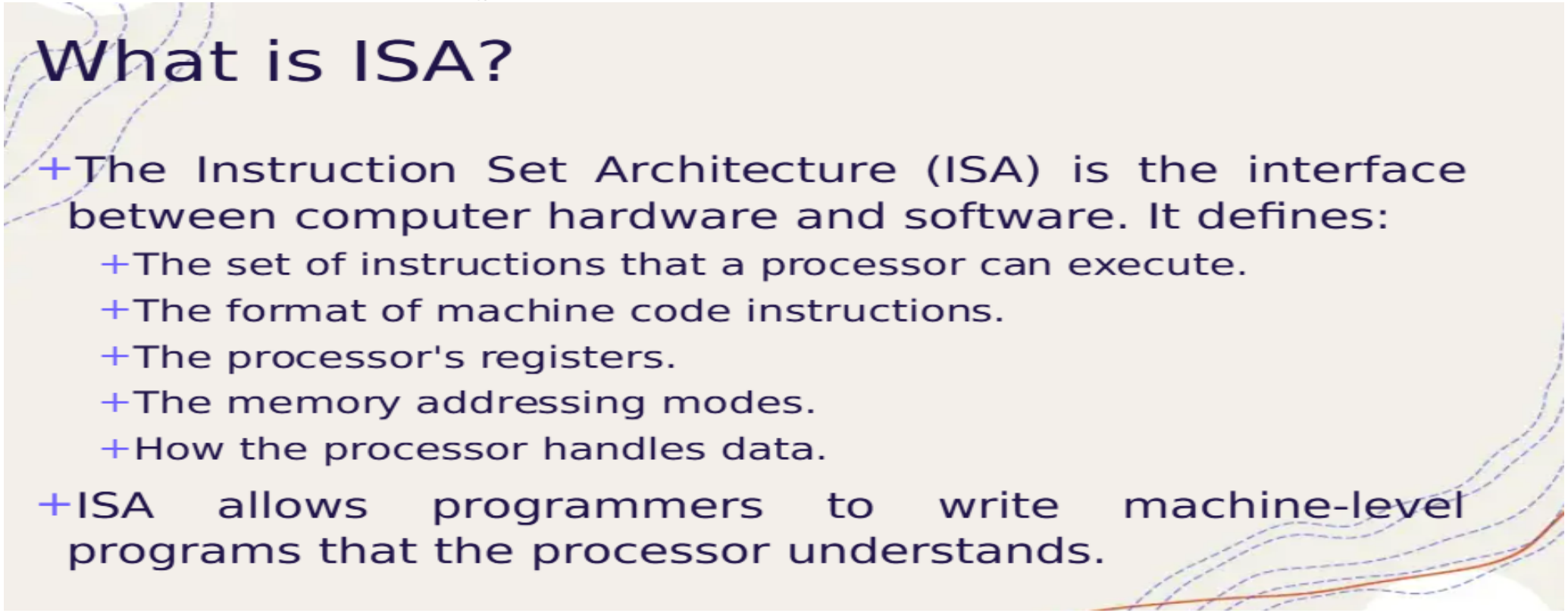
- **Instruction Set Architecture (ISA):**

  ISA is the language of the CPU that tells it what operations it can perform, such as adding numbers, loading data, or jumping to another instruction.
  It defines how software communicates with hardware through specific instruction rules and formats. It includes:

  - Instruction types (ADD, LOAD, JUMP), registers, data types, and memory access
  - Interrupt handling and system-level communication

ISA is the interface between software and hardware that defines the instructions and resources available to a processor.
Or
ISA is the language/rulebook understood by a processor.

- Instruction Set Architecture (ISA):
  - Concept of ISA



Computer Architecture =
Instruction Set Architecture
+ Machine Organization

Software-Hardware Interface: It acts as a translator model. Software writes code in machine or assembly language based on the ISA, and the hardware executes it.

Instruction Set Architecture

- Computer Architecture = Hardware + ISA

– Interface between all the **software** that runs on the machine and the **hardware** that executes it

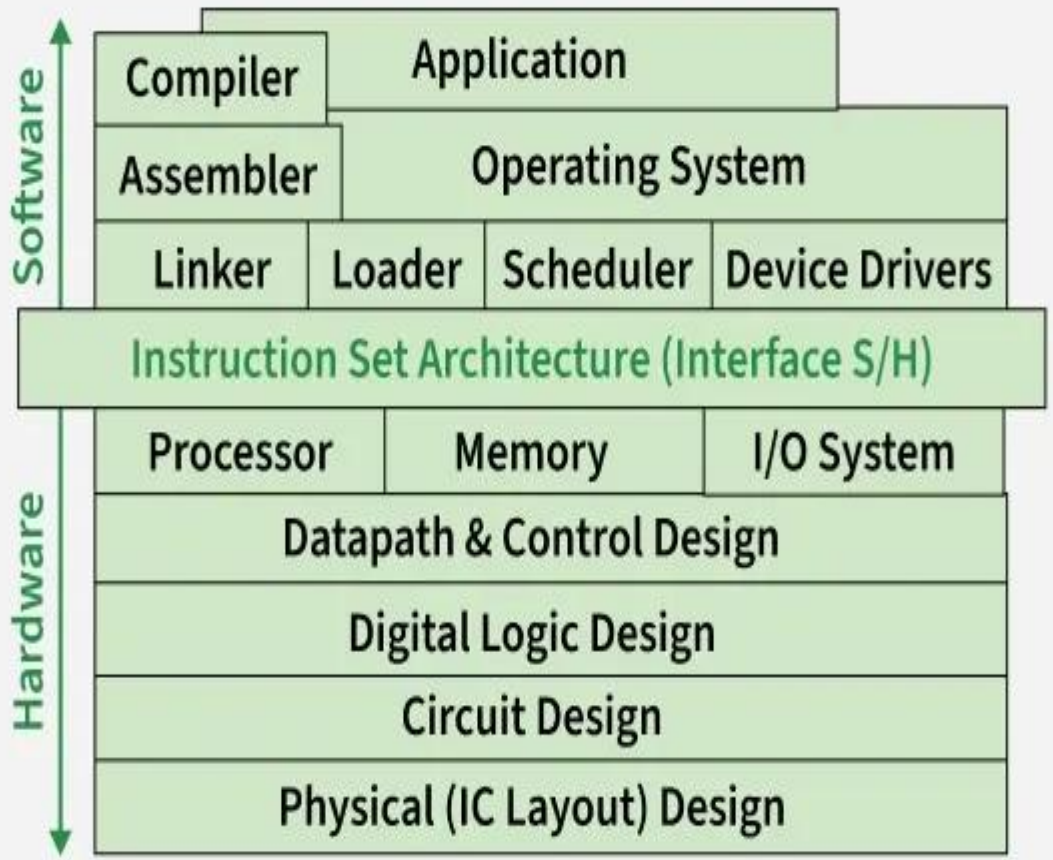
- An **instruction set**, or **instruction set architecture** (**ISA**), is the **part of the computer architecture related to programming**, including the native data types, instructions, registers, addressing modes, memory,
architecture, interrupt and exception handling, and external I/O. An ISA includes a specification of the set of opcodes (machine language), and the native commands implemented by a particular processor.

ISA acts as a bridge

Software → ISA → Hardware

- **Software:** Programs, compilers, operating systems
- **ISA:** Instruction set understood by the processor
- **Hardware:** ALU, registers, control unit, memory, buses

Instruction Set Architecture (ISA)



A. Physical (IC Layout) Design
This is the lowest level.
It deals with the physical implementation of electronic circuits on an integrated circuit (IC).

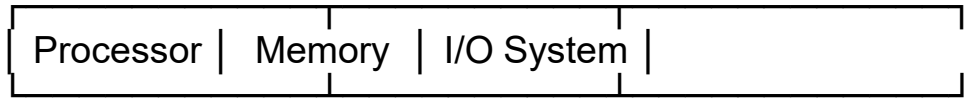
B. Circuit Design
At this level, engineers design electronic circuits that perform required functions.
For example:
Adders
Multiplexers
Registers
Flip-flops
Logic circuits

C. Digital Logic Design
These circuits are built using digital logic components such as:
AND
OR
NOT
XOR
Flip-flops
Registers
Counters

D. Datapath and Control Design
he **datapath** contains components through which data moves and is processed.

Processor, Memory and I/O System

Above the lower hardware-design layers, the diagram shows:



Processor

Executes instructions.

Memory

Stores:

Programs

Instructions

Data

I/O System

Allows the computer to communicate with external devices.

Examples:

Keyboard

Mouse

Display

Disk

Application
At the top is the **Application**.
Examples:
- Web browser
- Calculator
- MS Word
- Video player
- AI application
- Mobile application

Compiler
The **compiler** translates high-level programming language into lower-level/target code.

Assembler
The assembler converts **assembly language into machine code**.

Operating System
Examples:
Windows
Linux
Android
macOS

Linker
A **linker** combines different pieces of compiled/assembled code and libraries to create a final executable program.

Loader

The loader loads the executable program into memory so that the processor can execute it.

Scheduler

The scheduler is an operating-system component that decides which process/thread gets CPU time.

Device Drivers

A device driver is software that allows the operating system to communicate with a particular hardware device.

An ISA specifies several important characteristics of a processor.

Major components

1. **Instruction set**
    - Arithmetic instructions
    - Logical instructions
    - Data transfer instructions
    - Control-flow instructions
2. **Registers**
    - Number of registers
    - Register size
    - Special-purpose registers
3. **Data types**
    - Integer
    - Floating-point
    - Character
    - Boolean, etc.

4. Addressing modes

- Immediate
- Register
- Direct
- Indirect
- Indexed, etc.

5. Instruction formats

- Layout of bits within an instruction
- Opcode
- Operand fields
- Address/register fields

6. Memory organization

- Address size
- Memory addressing mechanism
- Alignment requirements

7. Exception and interrupt behavior

Key Components of ISA

- Instruction Set:
  - This is the collection of all the instructions that a CPU can execute.
  - These instructions can perform various operations, such as arithmetic, logical operations, data transfer, and control flow.
- Registers:
  - These are small, high-speed storage locations within the CPU that hold data and instructions that are currently being processed.
  - The ISA defines the number and types of registers available.

Key Components of ISA

- Memory Addressing Modes:
  - These determine how the CPU accesses memory locations.
  - Different addressing modes allow for flexibility in accessing data.
- Data Types:
  - The ISA defines the types of data that the CPU can handle, such as integers, floating-point numbers, and characters.

A **data type** specifies what kind of data a computer stores and processes.

Data Type	Short Explanation	Example
Integer	Whole numbers without decimal points	10, -25, 100
Floating Point	Numbers containing decimal values	3.14, 25.5
Character	A single letter, digit, or symbol	'A', '7', '@'
Boolean	Represents only two values: True or False	True, False

- int age = 20; → age stores an **integer**
- float temperature = 36.5; → temperature stores a **decimal number**
- char grade = 'A'; → grade stores a **single character**
- bool passed = true; → passed stores **True/False**

Key Components of ISA

- Instruction Formats:
  - This specifies the layout of the bits in an instruction, including the opcode (operation code) and operands (data or addresses).

Instruction format is the **arrangement of different fields/bits in a machine instruction**. It tells the processor **what operation to perform and on which data**.

Basic Instruction Format



Importance of ISA

1. Foundation of Processor Design
ISA forms the core design element of any processor. Whether it's RISC (Reduced Instruction Set Computing) or CISC (Complex Instruction Set Computing), the choice of ISA impacts all other design decisions.

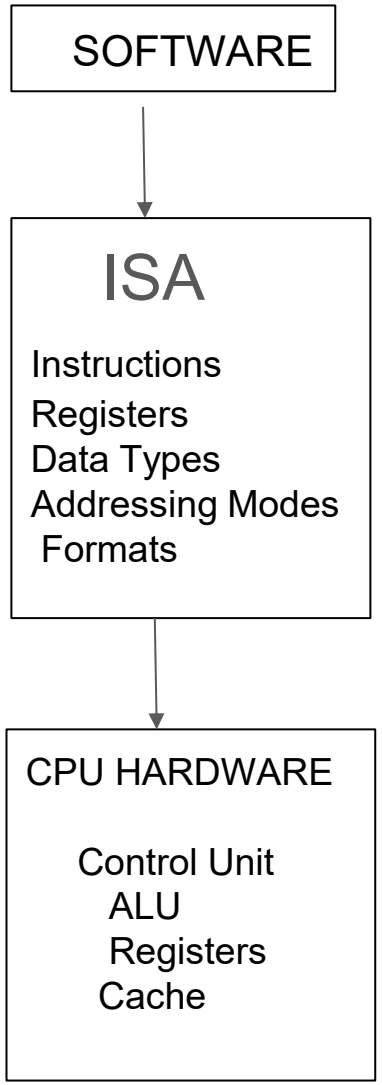
2. Instruction Execution Understanding
Computer architecture often focus on instruction execution, pipelining, control unit design, and instruction formats — all of which are defined by the ISA.

3. Enables Assembly Language Programming
Understanding ISA is critical for **assembly-level programming**. It helps in:
• Writing instruction sequences
• Understanding how data is loaded/stored
• Analyzing program execution time

4. Impact on Performance Metrics
A well-designed ISA can lead to efficient hardware implementation and optimized software execution. ISA affects:
• CPI (Cycles Per Instruction)
• Instruction count
• Execution time

5. Compatibility and Portability
ISA determines software compatibility. If two processors implement the same ISA, they can run the same programs — even if their internal microarchitectures are different.

Simple ISA Diagram



Example

If ISA specifies:

ADD R1, R2, R3

It means:

> Add the contents of R2 and R3 and place the result in R1.

The **organization** determines how the ALU, registers, buses and control unit actually perform this operation.

Basic Structure of an Instruction

A machine instruction generally consists of **three main parts**:



1. <u>Opcode</u> (Operation Code)

Specifies **what operation the CPU has to perform**.

Examples: ADD, SUB, LOAD, STORE

2. <u>Operand</u>

Specifies the **data or register** on which the operation is performed.

Example: R1, R2, 100

3. <u>Address</u>

Specifies **where the data is located** in memory.

Example: Memory address 2000

Example

ADD R1, R2

- **ADD** → Opcode → perform addition
- **R1, R2** → Operands → registers containing data

In simple words

Instruction = What to do + On what data + Where the data is located

Note: Not every instruction contains all three fields. The exact structure depends on the processor's **Instruction Set Architecture (ISA)**.

Basic Structure of an Instruction

A machine instruction generally contains:



Opcode

Opcode = Operation Code

It specifies the operation to be performed.

Examples:

ADD
SUB
MUL
LOAD
STORE
JUMP

Operand

Operands specify the data or locations on which the operation operates.

Example:

ADD R1, R2, R3

- `ADD` → Opcode
- `R1` → Destination operand
- `R2` → Source operand
- `R3` → Source operand

Types of Instructions

An **instruction** tells the CPU **what operation to perform**.

Instructions can be classified based on the operation they perform.

1. Arithmetic/Logic Instructions perform basic operations such as ADD, SUB, AND, and OR on data stored in registers.

2. Data Transfer Instructions are used to move data between memory and registers; for example, LW (load word) and SW (store word).

3. Branch and Jump Instructions control the execution flow of the program, making decisions and handling loops or function calls; examples include BEQ (branch if equal) and J (jump).

1. Data Transfer Instructions

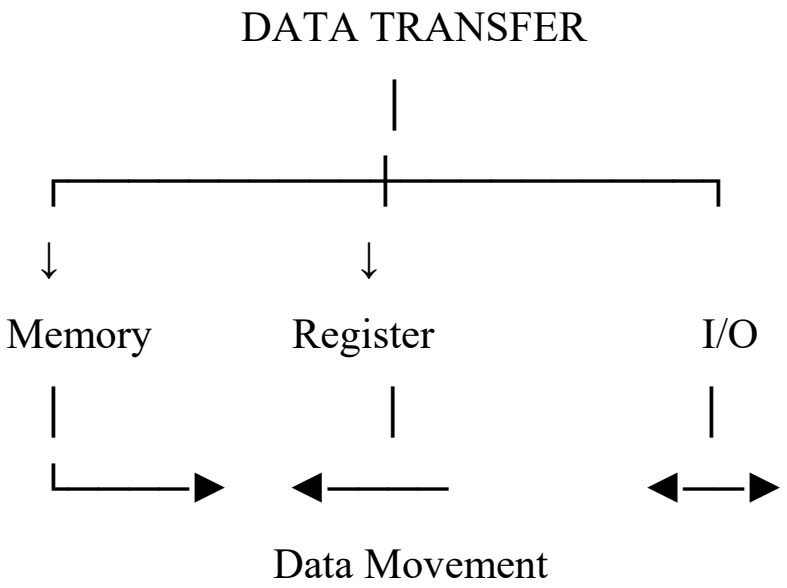
Data Transfer Instructions are instructions used to move data from one location to another inside the computer.

They do not normally change the actual value of the data; they only change where the data is stored.

Common operations

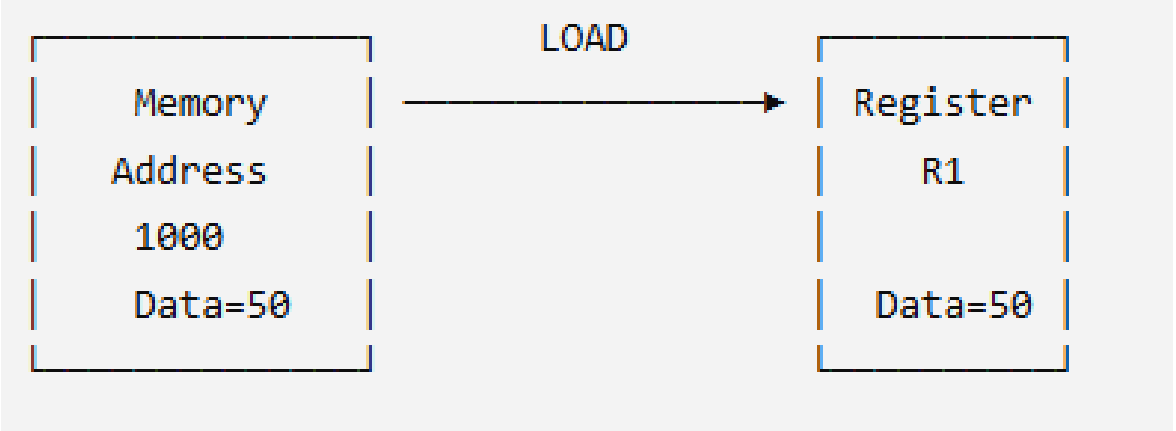
- LOAD
- STORE
- MOVE
- PUSH
- POP

Basic Concept



1. Memory → Register

Data is transferred from memory to a CPU register.



Example:

LOAD R1, 1000

Meaning:

Take the data stored at memory address 1000 and put it into R1.

If memory location 1000 contains 50:

Memory[1000] = 50

After execution:

R1 = 50

2. Register → Memory

Data is transferred from a **register to memory**.



Example:

STORE R1, 2000

Meaning:

Take the data from R1 and store it at memory address 2000.

If:

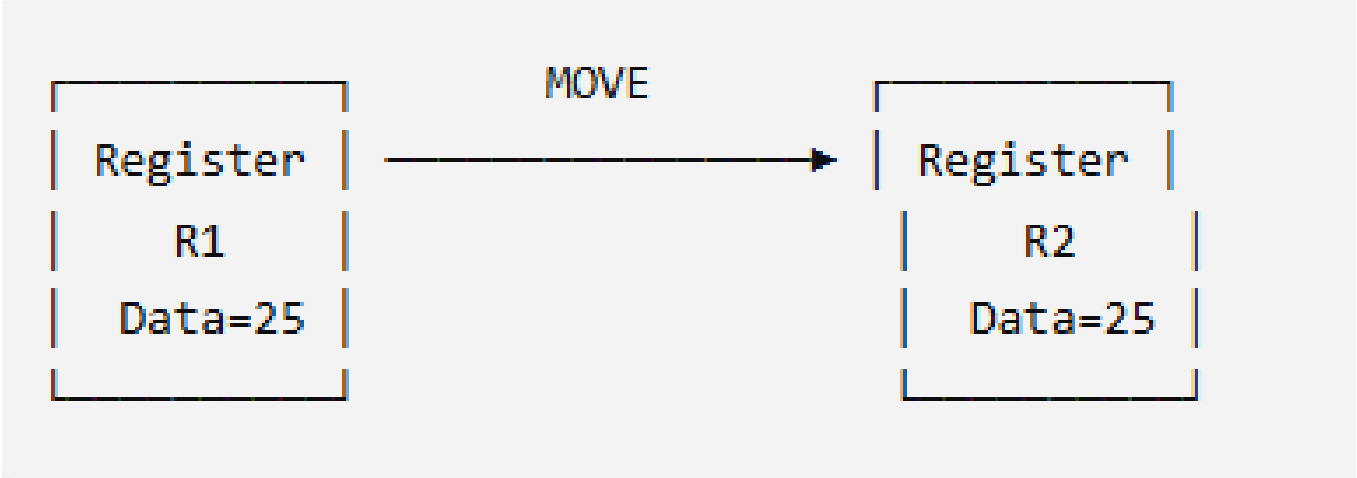
R1 = 50

After execution:

Memory[2000] = 50

3. Register → Register

Data is transferred from one **CPU register to another register**.



Example:
MOVE R2, R1
Meaning:
Copy the contents of **R1 into R2**.
If:
R1 = 25
R2 = 0
After execution:
R1 = 25
R2 = 25
Important: The original value in R1 remains unchanged.

Simple Example

Suppose:

Memory[1000] = 20

Memory[1004] = 30

Instructions:

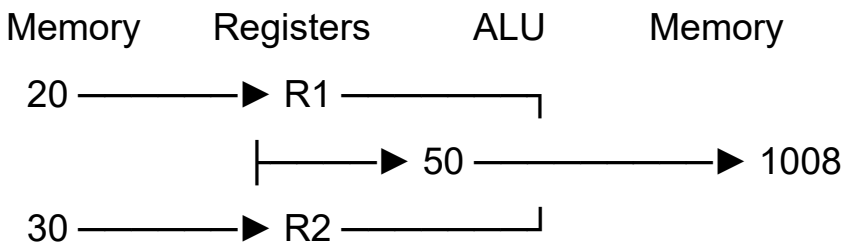
LOAD R1, 1000

LOAD R2, 1004

ADD   R3, R1, R2

STORE R3, 1008

Data movement is:



Here, **LOAD and STORE are data transfer instructions**, while **ADD is an arithmetic instruction**.

2. Arithmetic Instructions

Arithmetic instructions are instructions used by the CPU to perform **mathematical operations** on data.

Common arithmetic instructions

- ADD
- SUB
- MUL
- DIV
- INC
- DEC

Example

ADD R1, R2, R3

Meaning:

R1 = R2 + R3

Another example:

SUB R4, R5, R6

Meaning:

R4 = R5 - R6

3. Logical Instructions

Perform bit-level logical operations.

Common instructions

- AND
- OR
- NOT
- XOR

Example

AND R1, R2, R3

Meaning:

R1 = R2 AND R3

Shift and Rotate Instructions

Shift and Rotate instructions are used to move the bits (0s and 1s) of a binary number to the left or right.

Shift operations

- Logical Shift Left – LSL
- Logical Shift Right – LSR
- Arithmetic Shift Right – ASR

Rotate operations

- Rotate Left
- Rotate Right

Example:

SHL R1, 1

This shifts the bits of R1 one position to the left.

Shift Left
Bits are moved toward the **left**.

Example:
Original: 00101101
Shift left: 01011010

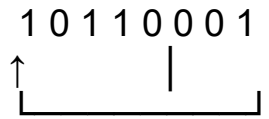
Diagram:

0 0 1 0 1 1 0 1
← ← ← ← ← ← ← ←
0 1 0 1 1 0 1 0

The leftmost 0 is discarded and a 0 enters from the right.

Rotate Left
Example:
Original: 10110001
Rotate left: 01100011

The leftmost 1 moves to the rightmost position.



0 1 1 0 0 0 1 1

Control Transfer Instructions

Control Transfer Instructions are instructions that change the normal sequence of execution of a program.

Normally, the CPU executes instructions one after another:

Instruction 1

↓

Instruction 2

↓

Instruction 3

↓

Instruction 4

↓

Instruction 5

A control transfer instruction can change this flow and make the CPU execute an instruction at another location.

Control Transfer Instructions

These instructions change the normal sequence of program execution.

Types

- Jump
- Branch
- Conditional branch
- Call
- Return

Example

JMP LOOP

The processor transfers control to the instruction labeled LOOP.

Conditional branch

BEQ LABEL

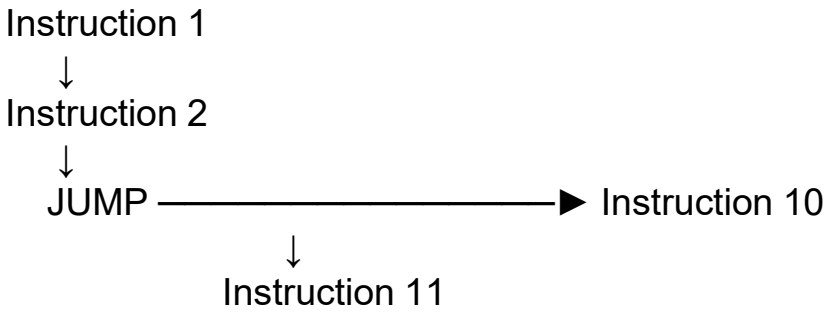
Meaning: Branch to LABEL if the specified condition is equal/true.

Control-transfer instructions are essential for:

- Loops
- If-else statements
- Function calls
- Decision making

Jump Instruction

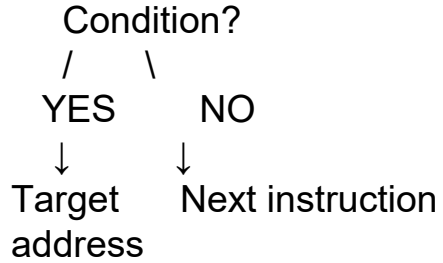
A **Jump** transfers control directly to another location.



Example
J LOOP
Meaning:
Go directly to the instruction labeled **LOOP**.
There is **no condition**.

Branch Instruction

A **Branch** transfers control **only when a specified condition is satisfie**



Example
BEQ R1, R2, LOOP
BEQ = Branch if Equal
Meaning:
If R1 = R2, go to LOOP; otherwise continue with the next instruction.

Input/Output Instructions

Used to communicate with I/O devices in architectures that provide explicit I/O instructions.

Examples

IN
OUT

They may be used for communication with:

- Keyboard
- Display
- Sensors
- Printers
- Other peripheral devices

Compare and Test Instructions

These instructions compare values or test particular conditions.

Examples

CMP R1, R2
TEST R1, R2

A compare operation generally affects **status/condition flags**, which can then be used by conditional branch instructions.

Example:

CMP R1, R2
BEQ SAME

Meaning:

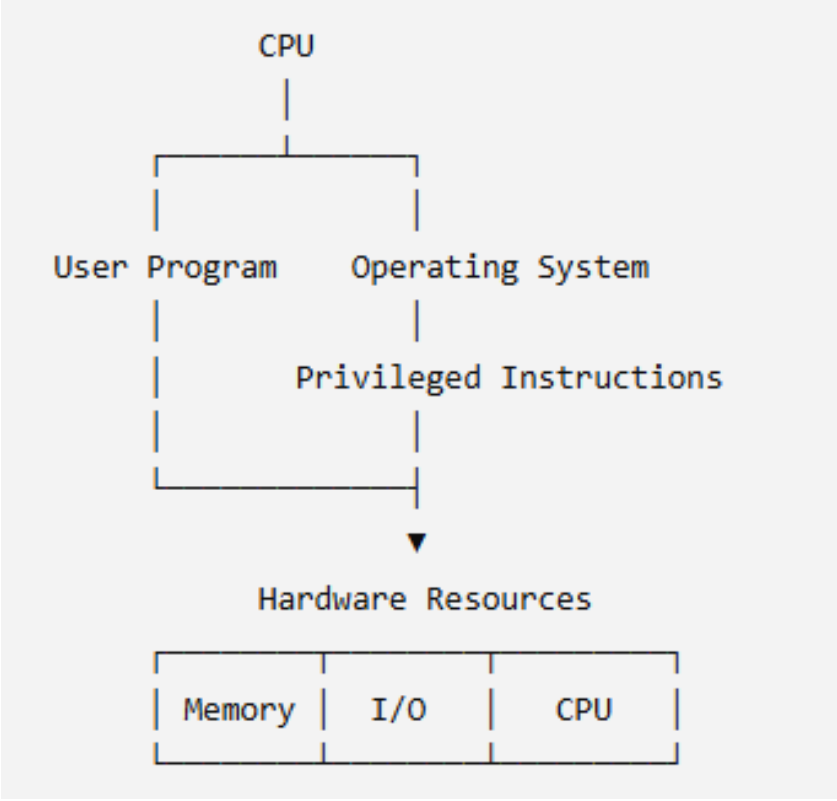
> Compare R1 and R2. If they are equal, branch to SAME.

System and Privileged Instructions

System and Privileged Instructions are special instructions used to control and manage the computer system. They are generally executed by the Operating System (OS) rather than ordinary user programs.

They provide access to important resources such as memory management, Managing interrupts, I/O devices, and processor control.

Basic Diagram



Why Are They Called Privileged?

Some operations are too important or dangerous to allow every program to perform directly.

For example, a normal application should not be allowed to:

- Directly control hardware devices
- Change memory-management settings
- Disable system interrupts
- Modify important CPU control registers

Therefore, the OS executes these operations in a privileged mode.

Instruction Length

MIPS is a 32-bit ISA, meaning every instruction must be exactly 32 bits (4 bytes) long. This fixed length simplifies the design and makes it more efficient for both hardware and compiler developers.

Instruction Formats

Since all MIPS instructions are 32 bits long, the ISA defines how those 32 bits are organized for different instruction types. MIPS uses three instruction formats:

Format	Used For
R-type	Arithmetic and logic operations (eg; ADD, SUB)
I-type	Data transfer and conditional branches (eg; LW, BEQ)
J-type	Unconditional jumps (eg; J)

Instruction Formats

An **instruction format** defines the arrangement of fields within a machine instruction.

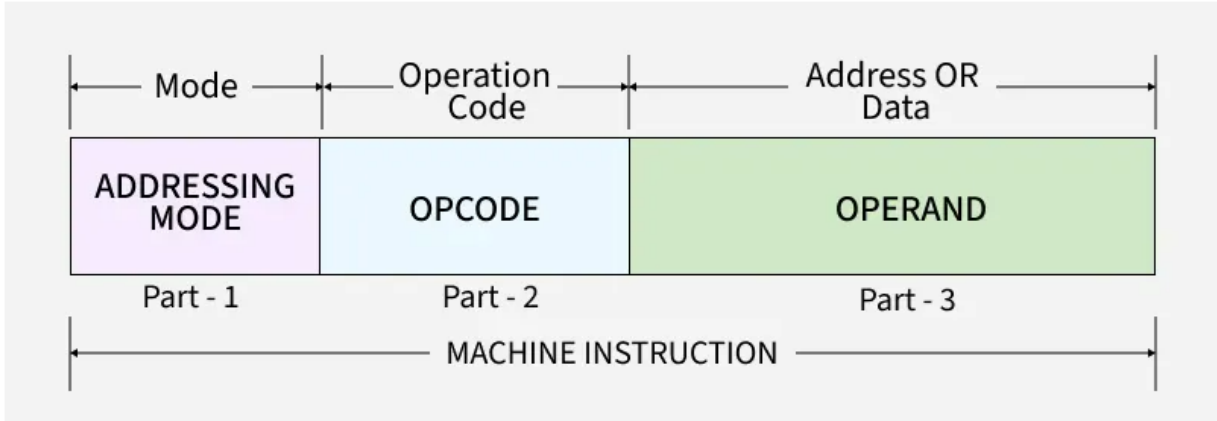
A general format is:



The exact format depends on the processor's ISA.

Typical fields

1. **Opcode**
2. **Register fields**
3. **Immediate value**
4. **Address/displacement**
5. **Function/sub-opcode fields**
6. **Other control bits**



- **Opcode:** This field specifies the operation to be performed by the CPU, such as addition, subtraction, or data transfer.
- **Operands:** These fields contain the data or references (addresses) to data on which the operation acts.
- **Addressing Mode:** This specifies how to interpret or locate the operand, such as direct, indirect, or immediate addressing.

Types of Instruction Formats

1. Three-Address Instruction

A three-address instruction contains three operand references.

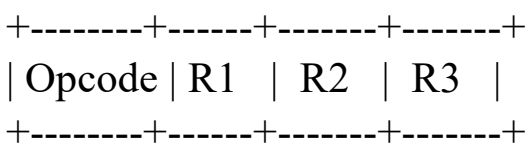
Example:

ADD R1, R2, R3

Meaning:

R1 = R2 + R3

Format



Advantage

It can preserve both source operands while storing the result separately.

2. Two-Address Instruction

A two-address instruction uses two operand fields.

Example:

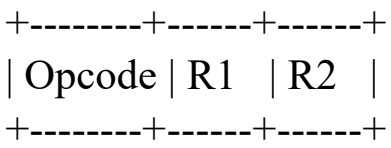
ADD R1, R2

Meaning:

R1 = R1 + R2

Here, R1 acts as both a source and destination.

Format



3. One-Address Instruction

A one-address architecture often uses an **accumulator** implicitly.

Example:

ADD X

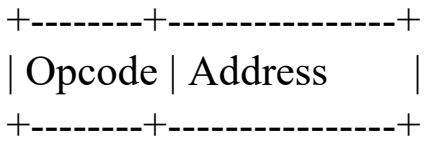
Meaning:

AC = AC + M[X]

Where:

- AC = Accumulator
- M[X] = Contents of memory location X

Format



4. Zero-Address Instruction

Zero-address instructions are commonly associated with **stack-based architectures**.

Example:

```
PUSH A
PUSH B
ADD
POP C
```

The ADD instruction does not explicitly specify operands because they are taken from the stack.

Conceptually:

Stack:

```
  B
  A
  ↓
 ADD
  ↓
A+B
```

Format	Address Fields	Typical Architecture	Example
Three-address	3	Register-based	ADD R1,R2,R3
Two-address	2	Register-based	ADD R1,R2
One-address	1	Accumulator-based	ADD X
Zero-address	0	Stack-based	ADD

Addressing Modes

An **addressing mode** specifies how the operand of an instruction is obtained.

1. Immediate Addressing

Operand is directly present in the instruction.

MOV R1, #10

Meaning:

R1 = 10

2. Register Addressing

Operand is stored in a register.

ADD R1, R2

The operands are obtained from registers.

3. Direct Addressing

Instruction contains the memory address of the operand.

LOAD R1, [1000]

Conceptually:R1 = M[1000]

4. Indirect Addressing

A register or memory location contains the address of the operand.

Conceptually:R1 = M[R2]

5. Indexed/Base + Offset Addressing

Effective address is calculated using a base register and an offset.

LOAD R1, 20(R2)

Conceptually:R1 = M[R2 + 20]

This is useful for:

- Arrays
- Structures
- Stack frames

Complete Instruction Execution

Consider:

ADD R1, R2, R3

Step 1 – Fetch

CPU fetches the instruction from memory.

Step 2 – Decode

Control unit identifies:

Opcode = ADD
Destination = R1
Source 1 = R2
Source 2 = R3

Step 3 – Read operands

CPU reads values from R2 and R3.

Step 4 – Execute

ALU performs:

R2 + R3

Step 5 – Write back

Result is stored in R1.

R1 = R2 + R3

Register Transfer Language and Micro operations

Register Transfer Language (RTL) is a symbolic shorthand notation used in computer architecture to describe the internal organization, data flow, and control sequences of digital systems.

Register Transfer Language (RTL) is a symbolic language used to describe:

- Transfer of data between registers
- Operations performed on data stored in registers
- Sequence of microoperations performed by the CPU

- A register is a group of flip-flops used to store binary information.
- RTL provides a simple way to describe what happens inside the processor at the register level.

Basic RTL notation

The basic notation is: R2←R1 , It means: Transfer the contents of register R1 into register R2.

Example

Suppose:

$$R_1 = 1010$$

Initially:

$$R_2 = 0000$$

After:

$$R_2 \leftarrow R_1$$

we get:

$$R_2 = 1010$$

while:

$$R_1 = 1010$$

remains unchanged.

Important RTL Symbols

Symbol	Meaning
R1,R2,R3	Registers
←	Transfer / assignment
+	Addition
–	Subtraction
+1	Increment
–1	Decrement
P:	Conditional operation
M[AR]	Memory contents at address stored in AR
T0,T1,T2	Timing signals

Micro-operation

- A micro-operation is an elementary, low-level operation performed on data stored within one or more registers during a single clock pulse.
- A microoperation is a basic operation performed on the data stored in registers. A microoperation is usually completed in one clock pulse.

Types of Micro-Operations

Micro-operations in digital computers are generally divided into four functional categories

1. Register Transfer Micro-Operations: Move binary data from one register to another without changing the data itself ($R_2 \leftarrow R_1$).

2. Arithmetic Micro-Operations: Perform basic mathematical operations on numeric data stored in registers:

- Addition: $R_3 \leftarrow R_1 + R_2$
- Subtraction: $R_3 \leftarrow R_1 - R_2$
- Increment: $R_1 \leftarrow R_1 + 1$

3. Logic Micro-Operations: Perform bitwise boolean operations on strings of bits:

- AND:
- OR:
- Complement

4. Shift Micro-Operations: Shift register data left or right by serial bit positions (logical, arithmetic, or circular shifts) used for scaling or bit manipulation:

- Shift right: $R_1 \leftarrow \text{shr } R_1$

Register Transfer

Register Transfer is the process of copying digital information or binary data from one computer register to another using hardware micro-operations.

Register transfer is the process of moving binary information from one register to another register within a digital computer.
It is represented using Register Transfer Language (RTL) notation.

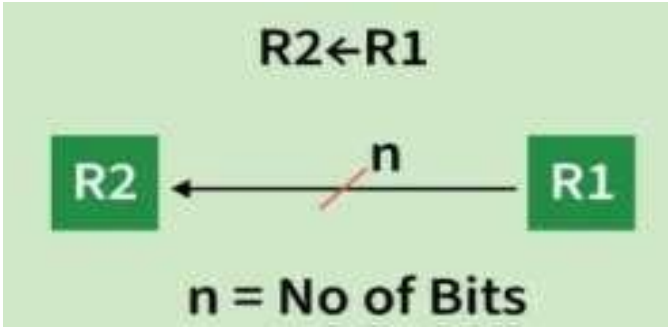
Basic notation:
R2 <- R1

Register Transfer Operations

The operation performed on the data stored in the registers are referred to as register transfer operations.
There are different types of register transfer operations:

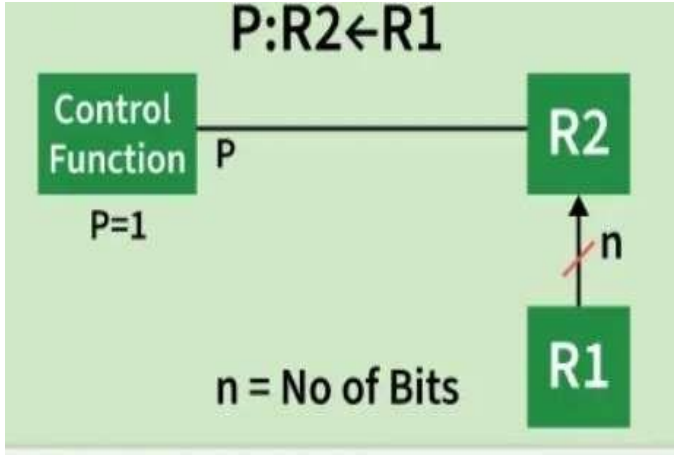
1. **Simple Transfer**

The content of R1 are copied into R2 without affecting the content of R1. It is an unconditional type of transfer operation.



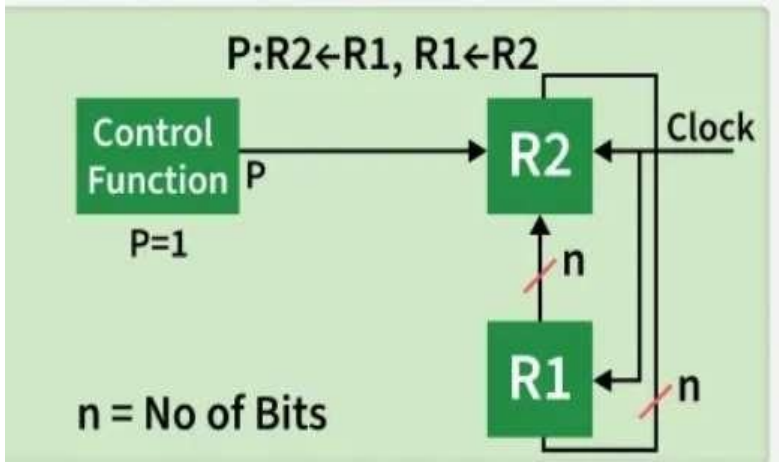
2. Conditional Transfer

It indicates that if P=1, then the content of R1 is transferred to R2. It is a unidirectional operation.



3. Simultaneous Operations

If 2 or more operations are to occur simultaneously then they are separated with comma **(,)**. If the control function P=1, then load the content of R1 into R2 and at the same clock load the content of R2 into R1.



Arithmetic microoperations

An arithmetic micro-operation is a basic, low-level hardware operation performed on numeric data stored in CPU registers

- **Addition:** $R_3 \leftarrow R_1 + R_2$ (Adds contents of R_1 and R_2, stores sum in R_3)
- **Subtraction:** $R_3 \leftarrow R_1 - R_2$ (Subtracts R_2 from R_1, stores difference in R_3)
- **Increment:** $R_1 \leftarrow R_1 + 1$ (Adds 1 to the value in R_1)
- **Decrement:** $R_1 \leftarrow R_1 - 1$ (Subtracts 1 from the value in R_1)
- **Complement (1's):** $R_1 \leftarrow \overline{R_1}$ (Inverts every bit in the register)
- **2's Complement:** $R_1 \leftarrow \overline{R_1} + 1$ (Inverts the bits and adds 1 to find the negative)

Binary Number: 1010011

Step 1: Find the 1's complement.

1010011 -> 0101100

Step 2: Add 1.

0101100
+ 1

0101101

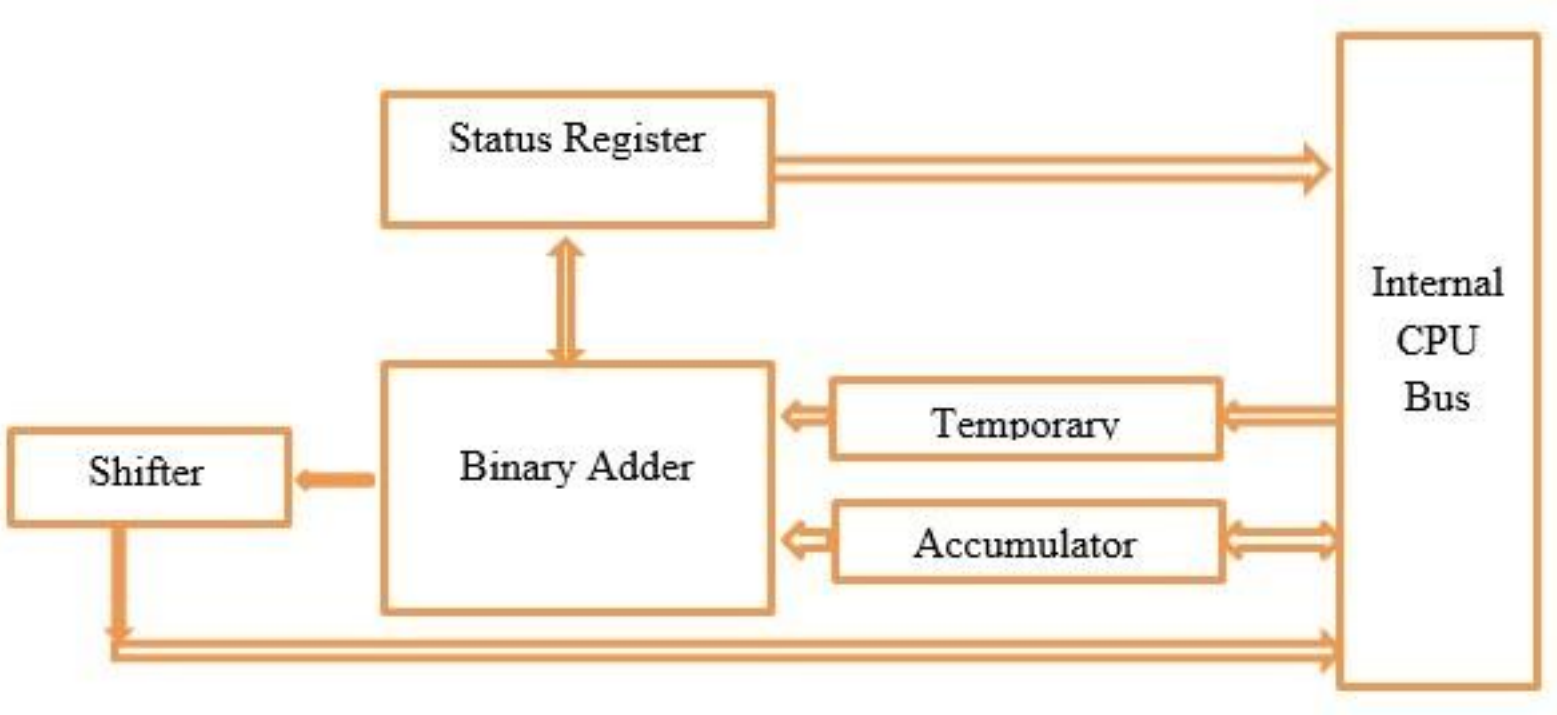
2's Complement = 0101101

ALU and its functions

- ALU (Arithmetic Logic Unit) is an important part of the Central Processing Unit (CPU). It performs arithmetic, logical, comparison, and bit-manipulation operations on data.

- The ALU receives data from CPU registers, performs the required operation, and stores the result back in a register or sends it to memory.

Block Diagram of ALU



ALU is an 8-bit unit

The ALU processes 8 bits of data at a time. Therefore, the registers and data paths connected to the ALU are designed to handle 8-bit values.

Main components of the ALU

The ALU shown in the figure contains:

1. Binary Adder
2. Shifter
3. Status Register

It also works with an Accumulator and a Temporary Register.

1. Binary Adder

The binary adder is the main arithmetic component of the ALU. It performs operations such as:

- Addition
- Subtraction
- Increment
- Decrement
- Other arithmetic operations using suitable control signals

2. Shifter

The shifter is used to shift or manipulate the individual bits of a data word.

Common operations include:

- Shift left
- Shift right
- Arithmetic shift
- Logical shift
- Rotate operations, depending on the processor design

3. Status Register

The Status Register stores information about the result produced by the ALU.

4. Temporary Register

The Temporary Register temporarily holds data coming from the internal CPU bus.

5. Accumulator

The Accumulator is a special register that stores an operand or an intermediate/result value, means that the value in the accumulator is added to the value supplied from the temporary register or CPU bus, and the result is returned to the accumulator.

Functions of ALU

The ALU mainly performs the following operations: Its major functions are:

1. **Arithmetic:** Addition, subtraction, increment, decrement, etc.
2. **Logical:** AND, OR, XOR, NOT.
3. **Comparison:** Equal, greater than, less than, etc.
4. **Shift:** Left and right shifts.
5. **Rotate:** Rotate bits left or right.
6. **Flag generation:** The ALU sets flags based on the result of operations to influence program control. Zero, Carry, Sign, Overflow, Parity, etc.

Stack-Based and General-Purpose Register Architectures

These are two important CPU instruction-set architectures based on how operands are stored and accessed during instruction execution.

1. Stack-Based Architecture
2. General-Purpose Register (GPR) Architecture

1. Stack-Based Architecture

In a stack-based architecture, operands are stored in a stack. The stack follows the LIFO (Last In, First Out) principle.

The CPU generally uses two main stack operations:

- PUSH – places data onto the stack
- POP – removes data from the stack

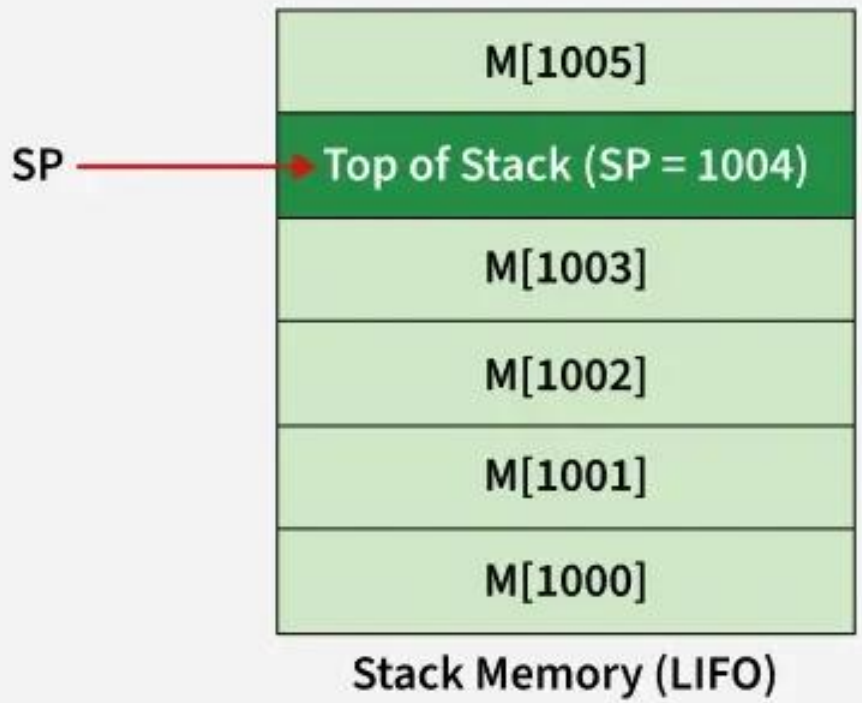
Arithmetic instructions usually operate on the top elements of the stack.

Example: C = A + B

```
PUSH A
PUSH B
ADD
POP C
```

Stack Pointer (SP) in Stack-Based CPU Organization

Push → SP Increments | Pop→ SP Decrements



Characteristics

- Uses an implicit stack for operands.
- Instructions are generally shorter because operand addresses need not be specified.
- Suitable for expression evaluation.
- Stack management can introduce additional PUSH/POP operations

Advantages

1. Simple instruction format.
2. Shorter instructions.
3. Easy evaluation of arithmetic expressions.
4. Fewer registers need to be explicitly specified

General-Purpose Register (GPR) Architecture

In a General-Purpose Register architecture, the processor contains several registers that can store operands,

intermediate results, and addresses.

Examples of registers:

R0, R1, R2, R3, ... Rn

For the expression:

C = A + B

The instruction could be

```
LOAD R1, A
LOAD R2, B
ADD R3, R1, R2
STORE C, R3
```

Here:

R1 ← A

R2 ← B

R3 ← R1 + R2

C ← R3

Characteristics

- Uses multiple registers for storing operands.
- Operands are explicitly specified in instructions.
- Provides fast access to frequently used data.
- Common in modern processors.

Advantages

1. Fast operand access.
2. Fewer memory accesses.
3. Suitable for complex calculations.
4. Flexible instruction execution.
5. Supports efficient compiler optimization.

Feature	Stack-Based	General-Purpose Register
Operand storage	Stack	Registers
Operand specification	Usually implicit	Usually explicit
Main operation	PUSH/POP	LOAD/STORE
Example	PUSH A; PUSH B; ADD	ADD R3,R1,R2
Instruction length	Usually shorter	Usually longer
Operand access	Top of stack	Any available register
Flexibility	Lower	Higher
Compiler optimization	Relatively difficult	Easier
Memory access	Can be higher	Generally lower
Typical use	Stack machines, JVM-like execution models	Modern CPUs

RISC and CISC Architectures

RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer) are two contrasting processor design philosophies that handle instruction execution and hardware organization differently.

RISC and CISC are two approaches to processor design with different instruction handling strategies.

- RISC uses a small set of simple, fixed-length instructions and follows a load/store approach, enabling efficient and fast execution.
- CISC uses a larger set of complex, variable-length instructions that can perform multiple operations, often requiring multiple clock cycles.

Reduced Instruction Set Architecture (RISC)

RISC simplifies processor design by using a small, uniform set of instructions. Each instruction performs a basic operation (e.g., load, compute, store) and is designed to execute in a single clock cycle, enabling efficient pipelining and simpler hardware.

Core principles of RISC architecture

The hallmark principles of RISC architecture are summarized in these five points:

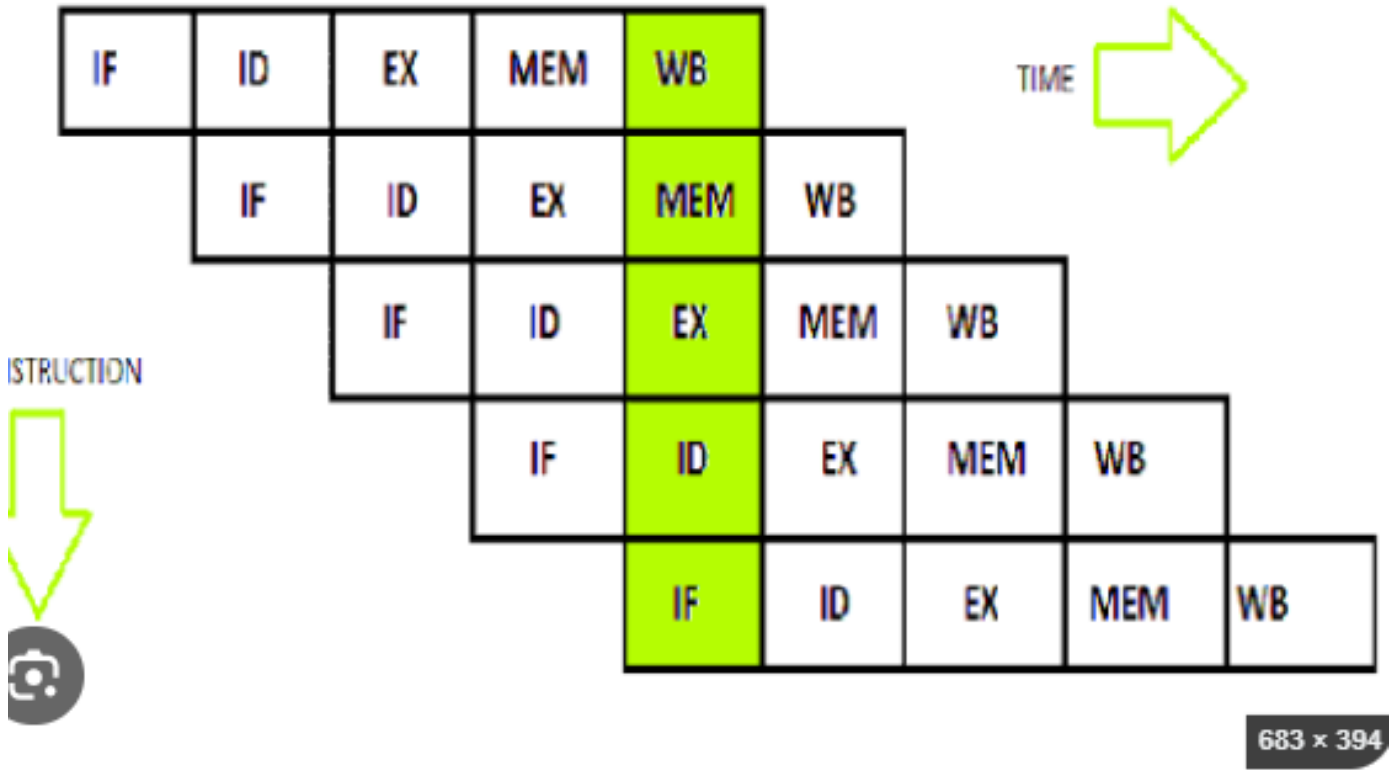
- **Single-cycle execution**: In most traditional central processing unit (CPU) designs, the peak possible execution rate is one instruction per basic machine cycle. For a given technology, the cycle time is determined by the slowest instruction in the instruction set. RISC designs aim to execute most instructions in a single cycle, increasing the processor's overall speed.

- **Load-store architecture:** RISC architectures use a load-store architecture, meaning only load and store instructions can access memory. All other instructions must operate on data in registers. This simplifies the instruction set and reduces the number of memory accesses required.

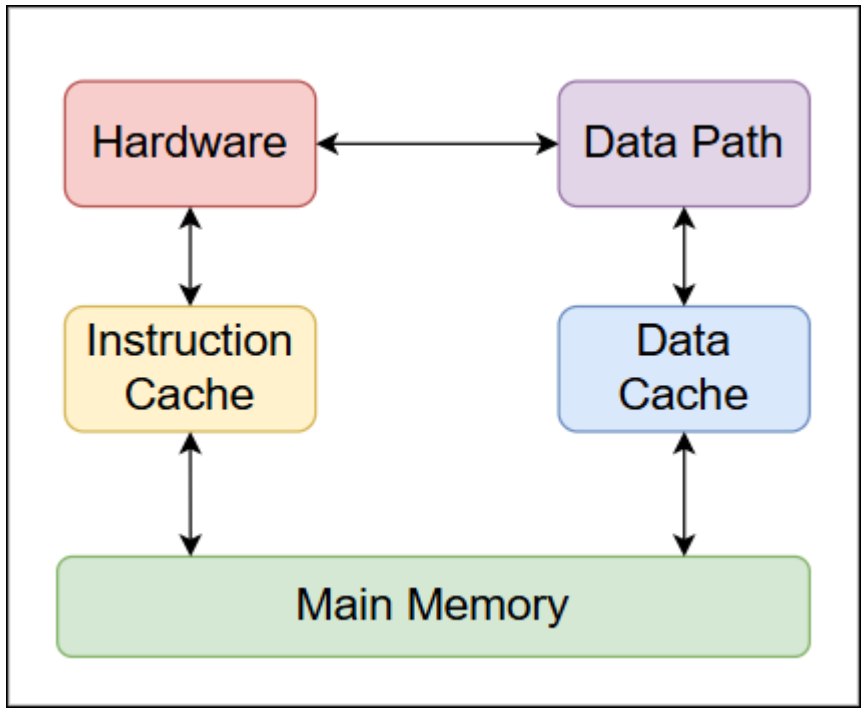
- **Simple instructions:** RISC architectures use simple instructions that can be executed quickly. This reduces the complexity of the processor and allows it to operate at a higher clock speed.

- **Large register set:** RISC architectures have many registers, which reduces the number of memory accesses required and allows for more efficient use of the processor's resources.

- **Pipelining:** RISC architectures use pipelining to increase the speed of instruction execution. Pipelining allows multiple instructions to be executed simultaneously, increasing the processor's overall throughput.


IF
ID
EX
MEM
WB
TIME
IF
ID
EX
MEM
WB
IF
ID
EX
MEM
WB
ISTRUCTION
IF
ID
EX
MEM
WB
IF
ID
EX
MEM
WB
683 × 394

RISC Architecture:



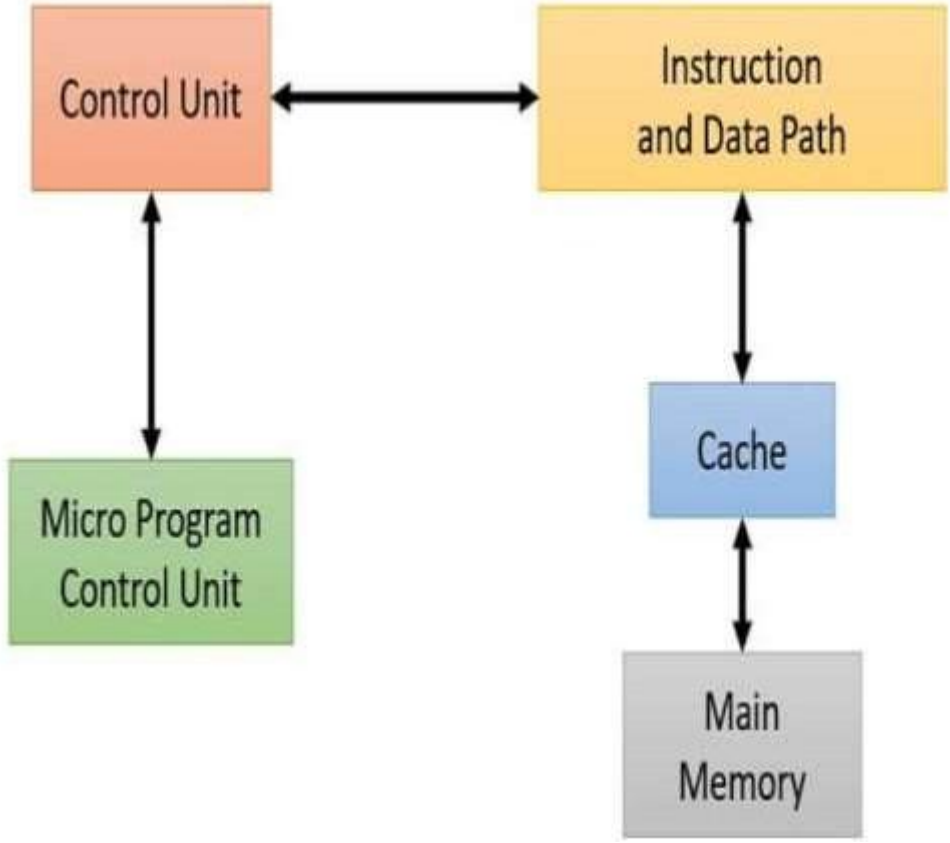
- The hardware of RISC architecture is designed to execute the instruction quickly, which is possible because of the more precise and smaller number of instructions and a large number of registers.

- In RISC, the data path is used to store and manipulate data in a computer. It is responsible for managing data within the processor and its movement between the processor and the memory.

- The processor uses a cache to reduce the access time to the main memory. The instruction cache is beneficial for retrieving and storing the data of frequently used instructions. It speeds up the process of instruction execution. The data cache provides storage for frequently used data from the main memory.

CISC Processors Architecture

- The main objective of a CISC processor is to minimize the program size by reducing the number of instructions in a program. This is done by 'embedding some of the low-level instructions in a single complex instruction'. Later when decoded this instruction generates several microinstructions to execute.
- Now if a program/software is getting simplified then the hardware has to get on work and must be able to perform the complex tasks. That's why a CISC processor has complex hardware. The block diagram of CISC architecture is given below:
- Here, you have a special microprogram control unit that uses a series of microinstructions of the microprogram stored in the "control memory" of the microprogram control unit and generate the control signals.
- The control units access the control signals produced by the microprogram control unit & operate the functioning of processors hardware.
- Instruction and data path fetches the opcode and operands of the instructions from the memory.
- Cache and main memory is the location where the program instructions and operands are stored.



Characteristics of CISC

- **Complex Instructions:** Features a large, rich set of specialized instructions capable of performing multiple low-level operations.
- **Variable-Length Encoding:** Instructions vary in size (often spanning multiple words) depending on complexity.
- **Multi-Clock Execution:** Complex instructions often require multiple clock cycles to complete.
- **Direct Memory Access:** Instructions can directly interact with and manipulate data inside main memory.
- **Fewer Registers:** Contains a smaller number of general-purpose registers as operations rely heavily on memory.
- **Microprogrammed Control:** Utilizes a microprogram control unit to translate complex macro instructions into sub-steps.

Feature	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Instruction Set	Small and simple	Large and complex
Instruction Size	Fixed-length (uniform)	Variable-length
Execution Time	1 clock cycle per instruction	Multiple clock cycles per instruction
Pipelining	Easy and highly effective	Harder and less efficient
Registers	Large number of general-purpose registers	Limited number of registers
Control Unit	Hardwired control unit	Microprogrammed control unit
Power Consumption	Lower power usage	Higher power consumption
Common Use Cases	Mobile devices, IoT, and embedded systems	Desktop processors and high-performance workstations

ISA designs are often discussed using the concepts of **RISC** and **CISC**.

RISC – Reduced Instruction Set Computer

Typical characteristics:

- Relatively simple instructions
- Large register use
- Load/store approach in many RISC designs
- Often regular instruction formats
- Designed to simplify efficient implementation

Examples include:

- ARM
- RISC-V

CISC – Complex Instruction Set Computer

Typical characteristics:

- Larger and more varied instruction sets
- More complex instruction encodings
- Some instructions can perform multiple low-level operations

Example:

- x86/x86-64

Types of ISAs

- CISC (Complex Instruction Set Computer):
  - have a large and complex set of instructions, some of which can perform multiple operations in a single instruction.
  - fewer instructions per program but more cycles per instruction
  - Examples: Intel x86 architecture, Motorola 68000.
  - Example Instruction (x86)

```
ADD AX, BX          ; Adds contents of BX to AX
```

Types of ISAs

- RISC (Reduced Instruction Set Computer):
  - These ISAs have a smaller and simpler set of instructions, each of which performs a single, basic operation.
  - More instructions per program but fewer cycles per instruction.
  - Examples: ARM architecture, MIPS architecture, RISC-V.
  - Example Instruction (MIPS)

```
ADD $t0, $t1, $t2          ; Adds $t1 and $t2, stores
in $t0
```

Types of ISAs

- VLIW (Very Long Instruction Word):
  - Executes multiple instructions in parallel
  - Used in high-performance computing
  - contains instructions that encode multiple operations within a single word, allowing for parallel execution of these operations
  - Example: Intel Itanium, Transmeta Crusoe

Types of ISAs

- EPIC (Explicitly Parallel Instruction Computing):
  - Similar to VLIW but with more parallelism control
  - Example: Intel IA-64 (Itanium)

ISA Examples from Different Processors

Processor	ISA Type	Example Instruction
Intel x86	CISC	`MOV AX, 1000H`
ARM	RISC	`ADD R0, R1, R2`
MIPS	RISC	`LW $t0, 0($t1)`
8085	CISC	`LDA 2050H`